



THE UNIVERSITY
OF QUEENSLAND
AUSTRALIA

CREATE CHANGE

Advanced Database Systems (INFS3200)

Lecture 2: Distributed Database Design

Lecturer: Dr Junliang Yu

School of Electrical Engineering and Computer Science (EECS)

Faculty of Engineering, Architecture and Information Technology

The University of Queensland

Examples of DDB



Google Spanner



Apache Cassandra



CockroachDB



Amazon DynamoDB



Microsoft CosmosDB



Apache HBase



Nebula Graph

Too many DDBs...

Questions in Mind

What are distributed databases?

Why do we need distributed databases?

What are the benefits of using distributed databases?

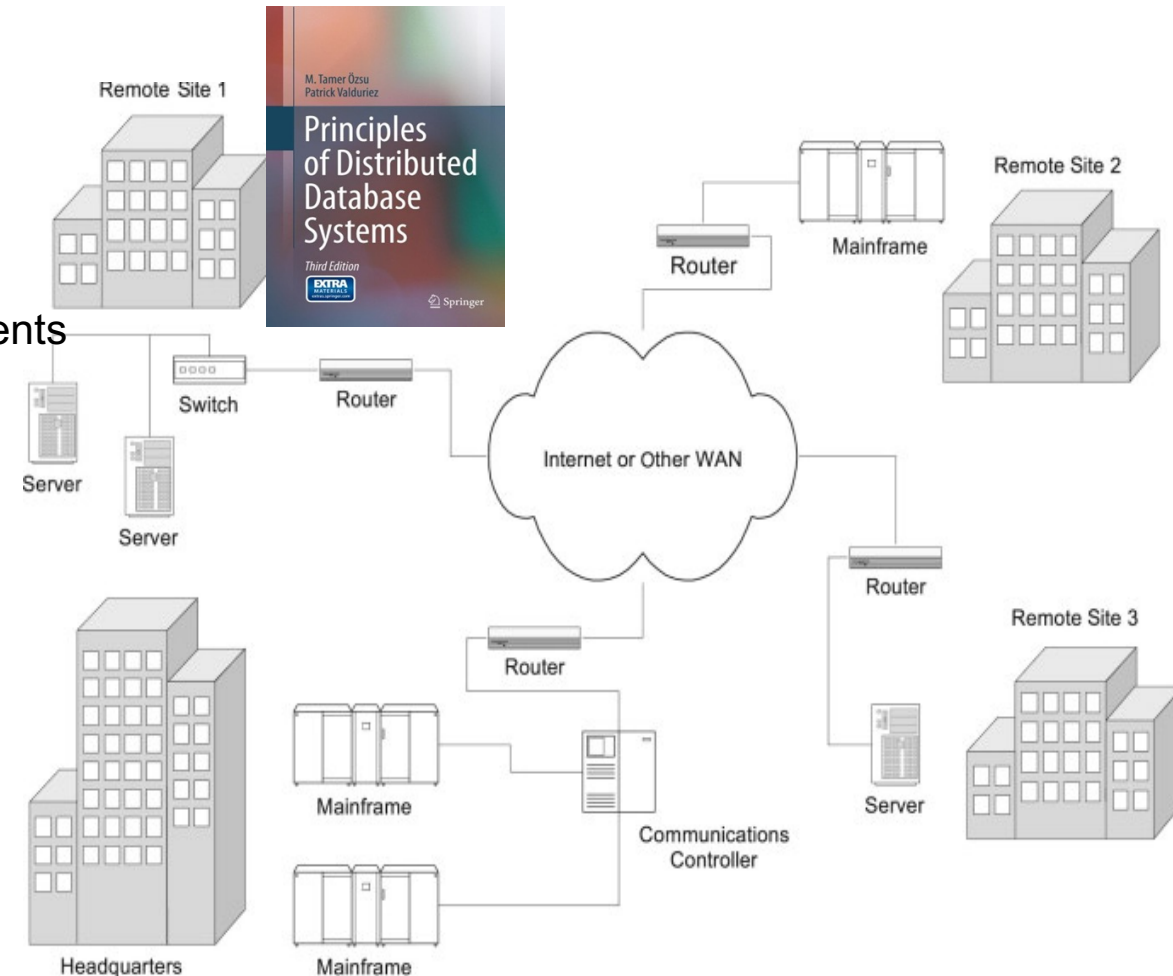
How to design a distributed database?

Centralized DB vs. Distributed DB

Feature	Centralized Database	Distributed Database
Definition	A single database stored in a central location.	A database spread across multiple locations or nodes.
Storage Location	Stored in a single physical site.	Stored across multiple sites or nodes.
Data Access	Accessed from a single server.	Accessed from multiple servers.
Reliability	Lower reliability; a single point of failure.	Higher reliability; failure of one node does not impact the system.
Scalability	Limited scalability; performance degrades with load.	High scalability; additional nodes can be added to handle load.
Data Redundancy	Minimal redundancy; typically a single copy of data.	High redundancy; multiple copies of data improve fault tolerance.
Data Consistency	Easier to maintain consistency as all data is in one place.	More complex to maintain consistency due to data replication.
Response Time	May be slower under heavy loads due to a single access point.	Faster response time in distributed environments.
Security	Easier to secure as data is stored in one location.	More challenging due to data distribution across multiple locations.

Outline

- Concepts about Distributed Systems
- Motivations and Benefits of using DDBs
 - Types of Transparency, Reliability and Availability
 - Performance Issues, Expansion and Scalability
- Concepts in DDBs, DDB Architecture: Schemas and Components
- DDB Design
 - Factors to be considered
 - Two Approaches - Top-Down Process with an example
 - Fragmentation Properties
 - Minterm Predicates Approach
 - Derived Horizontal Fragmentation and Semi-Join
 - Vertical Fragmentation and Properties
 - Allocation of Fragments & SSOT Property



Concepts about Distributed Systems

- Distributed computing systems
 - A number of **autonomous** processing elements, not necessarily **homogeneous**, that are **interconnected** by a computer network and that cooperate in performing their assigned tasks
 - E.g. Google Search Engine and Apache Hadoop
- Distributed database (DDB)
 - A collection of multiple **logically interrelated** databases distributed over a computer network with a global schema.
- Distributed database management system (DDBMS)
 - A software that manages a distributed database while making the distribution **transparent** to the user

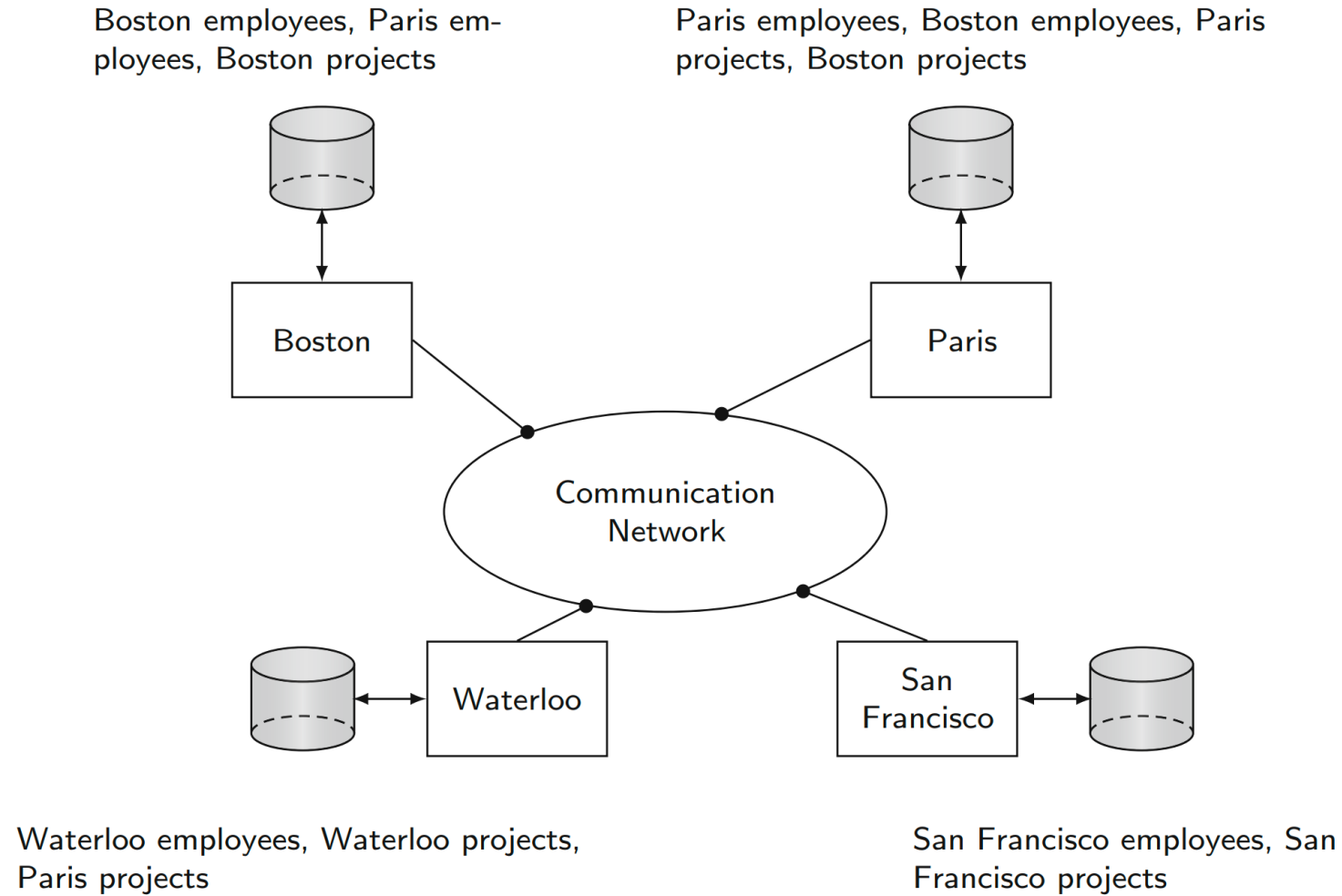


Fig 1.5 in the textbook Principles of Distributed Database Systems (4th Edition)

Why Distributed?

- Corresponds to the organizational structure of **distributed enterprises**
 - Physically distributed **companies**
 - E-commerce companies with different **geographical centers**
 - **Supply chain** management
- Economic benefits
 - Easy to manage - **local autonomy**
 - Able to share – **global dictionary (schema)**, data integrity management, transaction management
 - More computing power - **processing efficiency** and **minimization of data transmission costs**
- Key characteristic: **work towards a common goal**
 - Distributed systems (Dropbox/OneDrive, Amazon) vs parallel systems (HPC)

Benefits of DDBMS

Transparency

- Users don't need to know...

Reliability

- When something fails...

Availability

- When something is needed...

Performance

- Do things faster...

Expansion

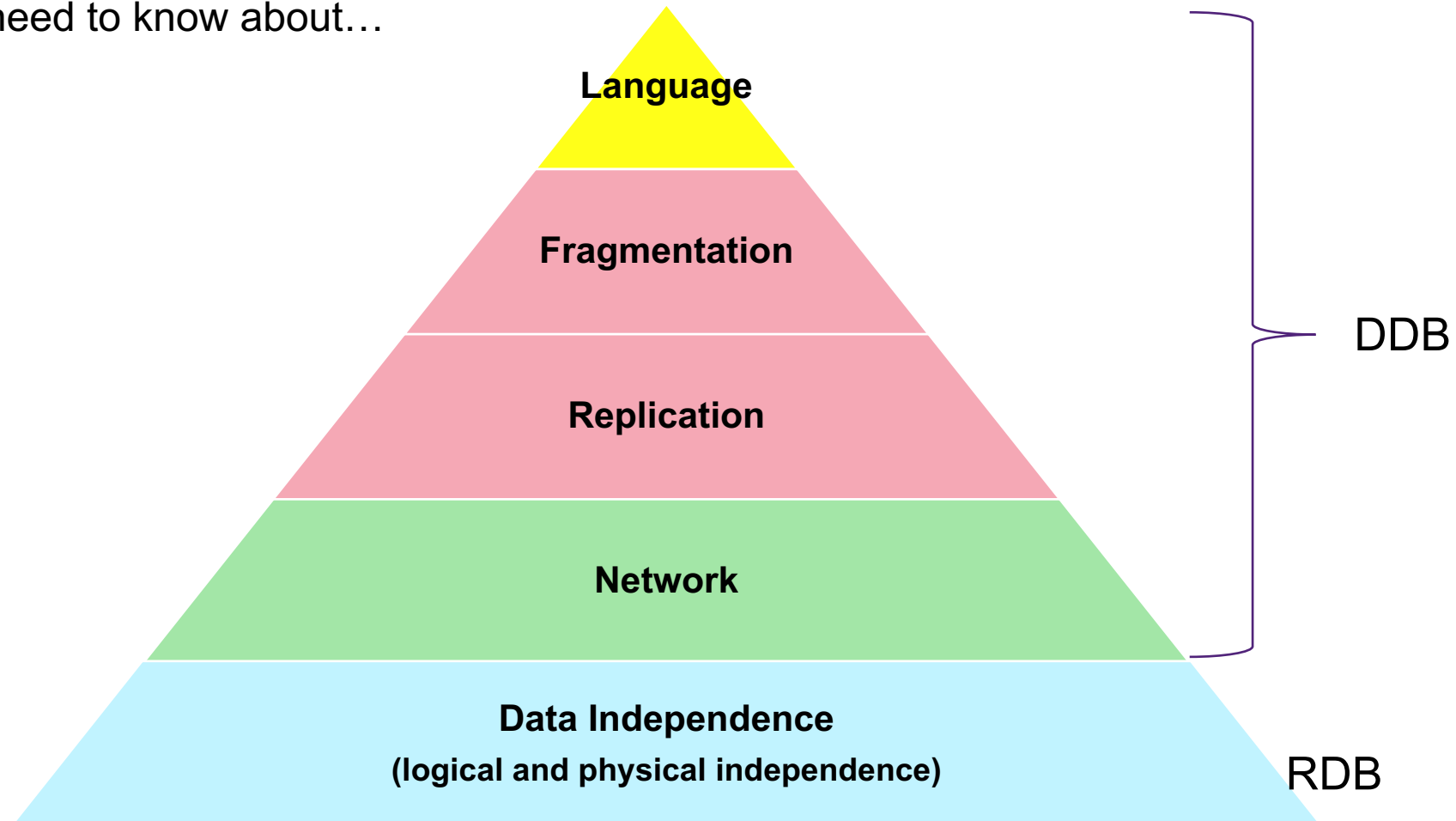
- Add more resources later...

Data Integrity

- Is DDB still a DB?

Many Types of Transparency

Users don't need to know about...



Reliability and Availability

Resources are **more available in a reliable** system

- Poor reliability and poor availability because of faults, errors and failures, which can be caused by many reasons in a DBMS

Replicated components (data & software) eliminate **single point of failure**

- Failure of a single site or communication link should not bring down the entire system (*fault tolerance*)
- Managing the 'reachable' data requires **distributed transaction support** which provides correctness in the presence of failures

...good reliability $\Rightarrow$ good availability?

...poor reliability $\Rightarrow$ poor availability?

Performance Issues

Data localization

- Reduces contention for CPU and I/O services
- Reduces communication overhead

Parallel processing

- Inter-query & intra-query parallelism

In general, performance can be achieved in many other ways (indexing, query optimization, or SQL Chip)



Expansion and Scalability

Scale-Up: Expand the capacity of system by upgrading with larger storage cabinets or supercomputers

Scale-Out: Expand the capacity of system by adding more computing processor nodes and memory and storage nodes to the network



...elasticity
...scale-up vs. scale-out

When to Distribute?

Distribution is **expensive** to manage

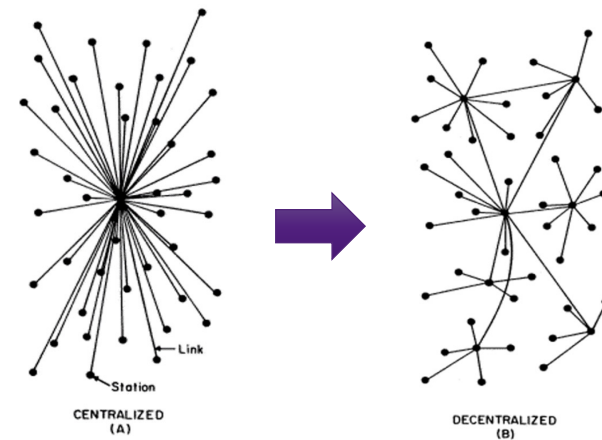
If you want to control the whole system, centralization in general is a better idea

- But distribution is still desirable for handling **very large data volumes and distributed data accesses**

Distribute when you don't control the whole system

- Localized data access
- Mobile computing
- Systems controlled by organisational units

DDB theory essential to making an **informed decision**



Concepts in Distributed Databases

Distribution

- Whether the components of the system are located on the same machine or not (distribution vs. centered)

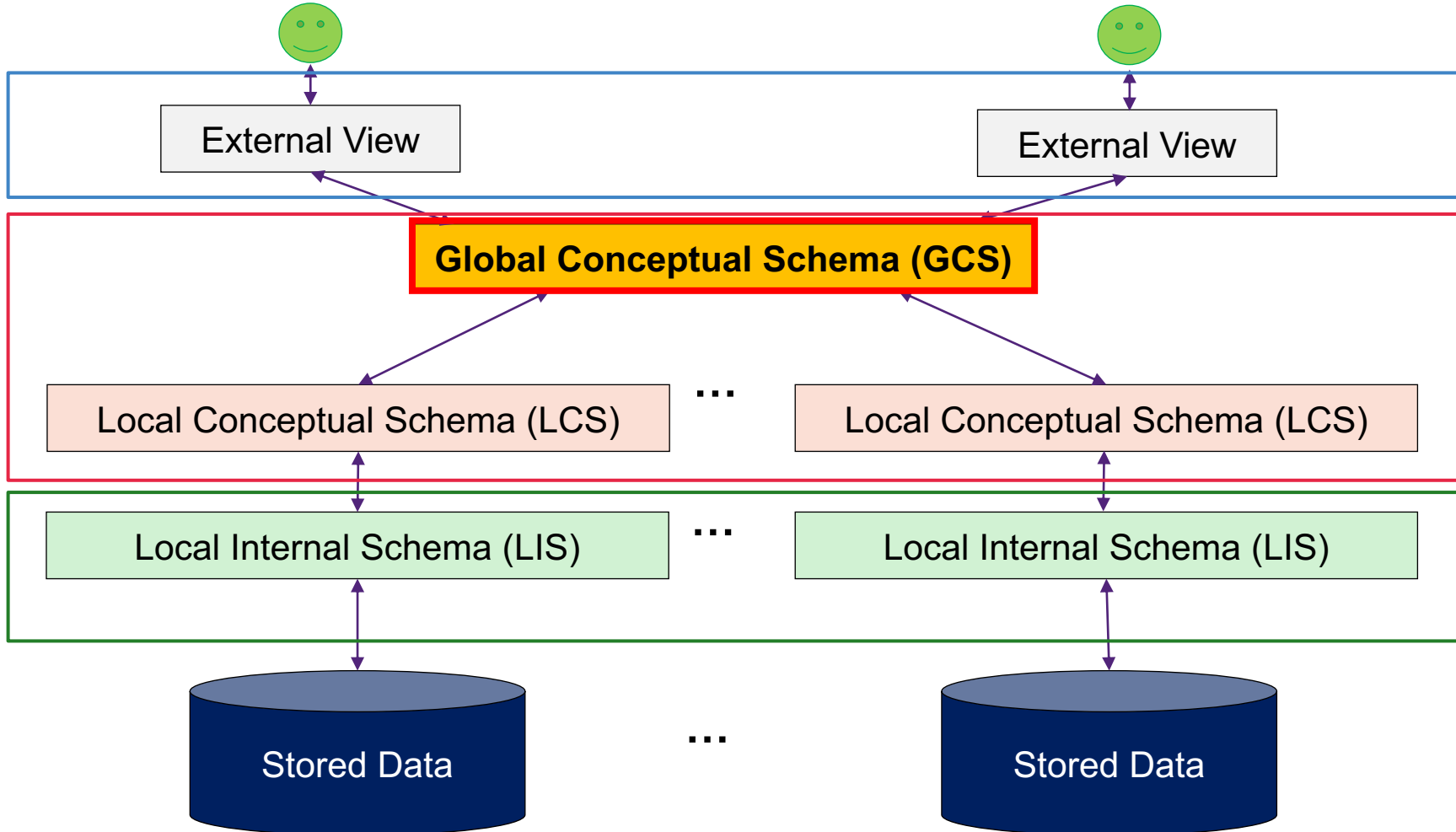
Homogeneity

- Homogeneous: every site runs the same type of DBMS
- Heterogeneous: different sites can run different DBMSs (different RDBMSs or even non-relational DBMSs)

Autonomy

- If each node can operate independently (*local autonomy*)
- Different types of autonomy:
 - Design autonomy, sharing autonomy and execution autonomy

DDB Architecture: Schemas



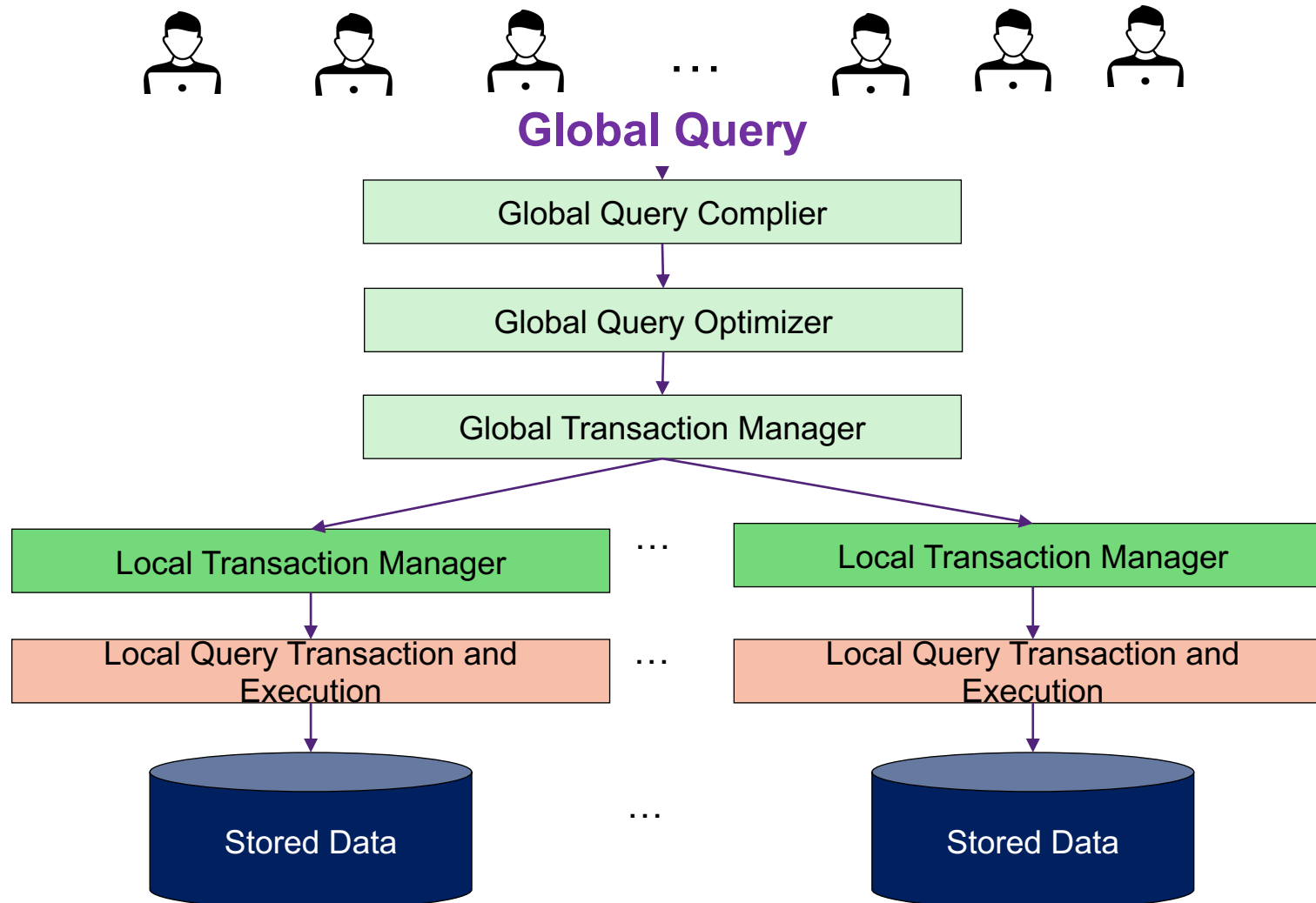
ANSI/SPARC
DBMS model

External Level

Conceptual Level

Internal Level

DDB Architecture: Components



DDB Design: Factors to be Considered

Design of the **network**

Distribution of the **nodes**

Distribution of **application software**

Distribution of data

- **Level of sharing**
- **Access patterns**
- **Level of knowledge about site databases...**

Distributed Database Design

The **Top-down** Approach – *a design approach*

- Starting with the design of GCS (**Global Conceptual Schema**)
- Considering data **fragmentation** and **allocation**
- Designing component databases

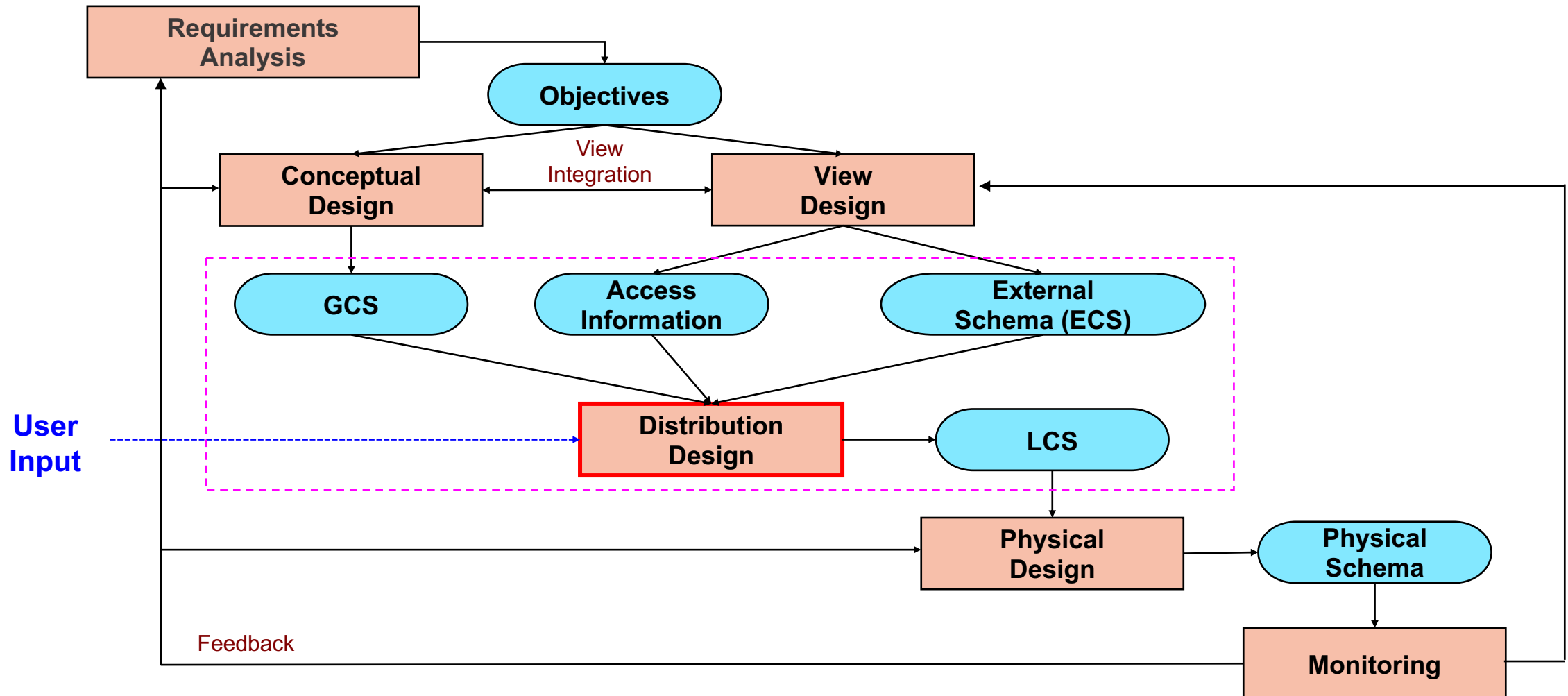
The **Bottom-up** Approach – *an integration approach*

- Integration of **pre-existing LCSs** into a **GCS**
 - Design views and external schemas
 - Local systems will most likely be heterogeneous
- Many problems with integration and interoperability
 - Often more business problems than technical problems
- May lead to database integration, or federated databases

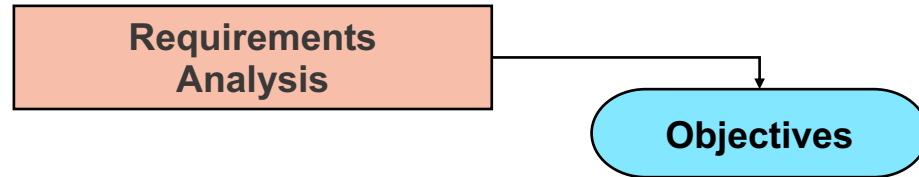
Which one
is better?

...more about data integration later

Design Process: Top-Down



An Example of Top-Down Design Process (1)

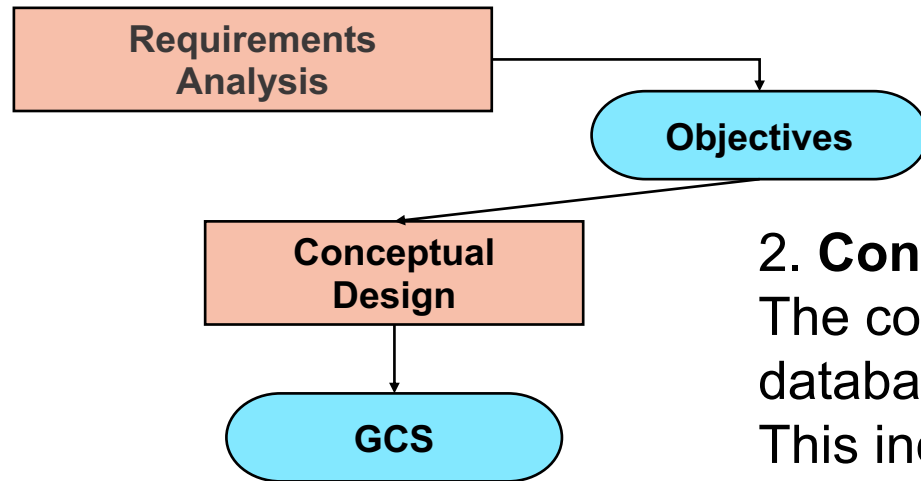


1. Requirement Analysis:

In this phase, we determine the needs and expectations of the database system. For Woolworths, the system needs to:

- Handle transactions from hundreds of stores nationwide.
- Update inventory in real time.
- Provide robust analytics for stock levels, sales trends, and supply chain logistics.
- Ensure data consistency and high availability.
- Offer scalability to accommodate future growth.

An Example of Top-Down Design Process (2)

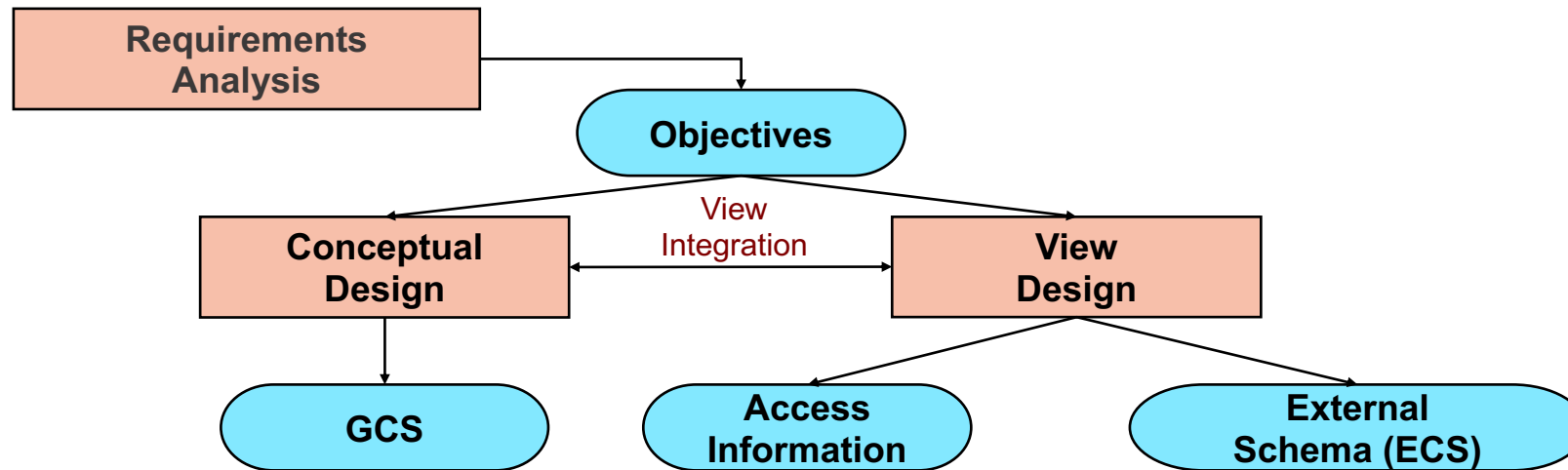


2. Conceptual Design

The conceptual design creates a unified view of the entire database, regardless of how or where data will be distributed. This includes defining:

- **Data entities** such as products, stores, employees, sales, and suppliers.
- **Relationships** between entities, like sales transactions linked to specific store locations and products.
- **Constraints** and data integrity rules, such as product stock levels, cannot be negative.
- An **Entity-Relationship (ER) diagram** will be developed to visualise the conceptual model (GCS).

An Example of Top-Down Design Process (3)

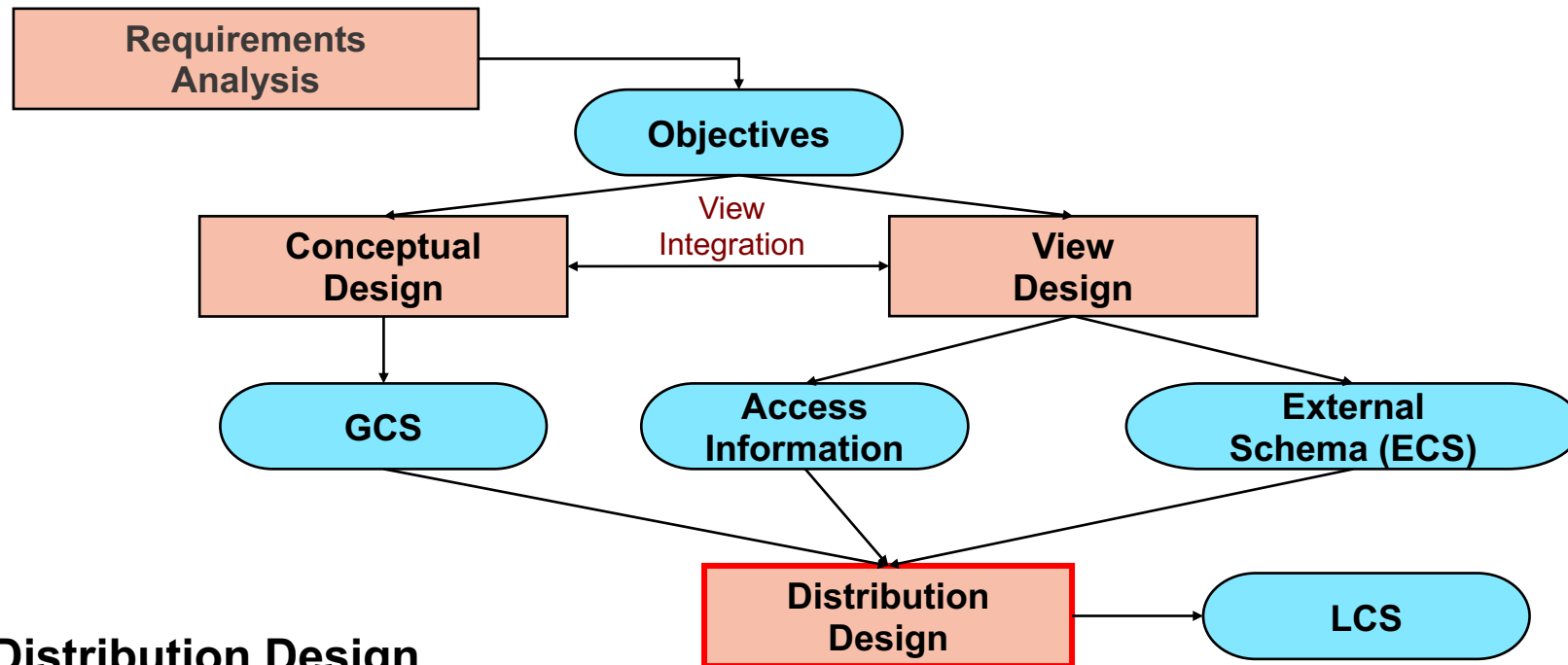


3. View Design

View design involves creating the user's perspective of the database. It defines how different stakeholders interact with the system:

- **Store managers** view inventory and sales data for their specific location.
- **The supply chain team** accesses supplier data and cross-store inventory levels.
- **The analytics team** uses sales data across all stores for trend analysis.

An Example of Top-Down Design Process (4)



4. Distribution Design

Here, we decide how to distribute the database across multiple sites. There are two key sub-tasks:

- **Data Fragmentation:** Dividing the data into fragments. Woolworth might fragment data by region, with each regional server holding data for its stores.
- **Data Allocation:** Assigning fragments to particular sites. Woolworth's system would place data close to where transaction processing occurs to reduce latency and network load.

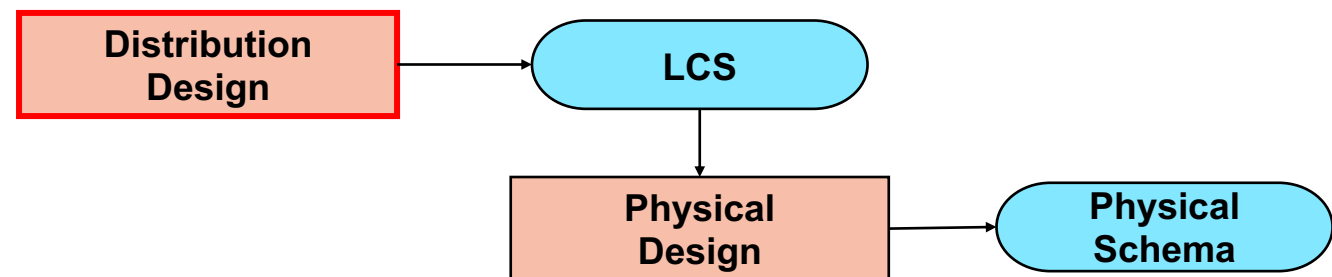
An Example of Top-Down Design Process (5)

5. Physical Design

Physical design involves defining the actual storage structure and access methods for the database, including:

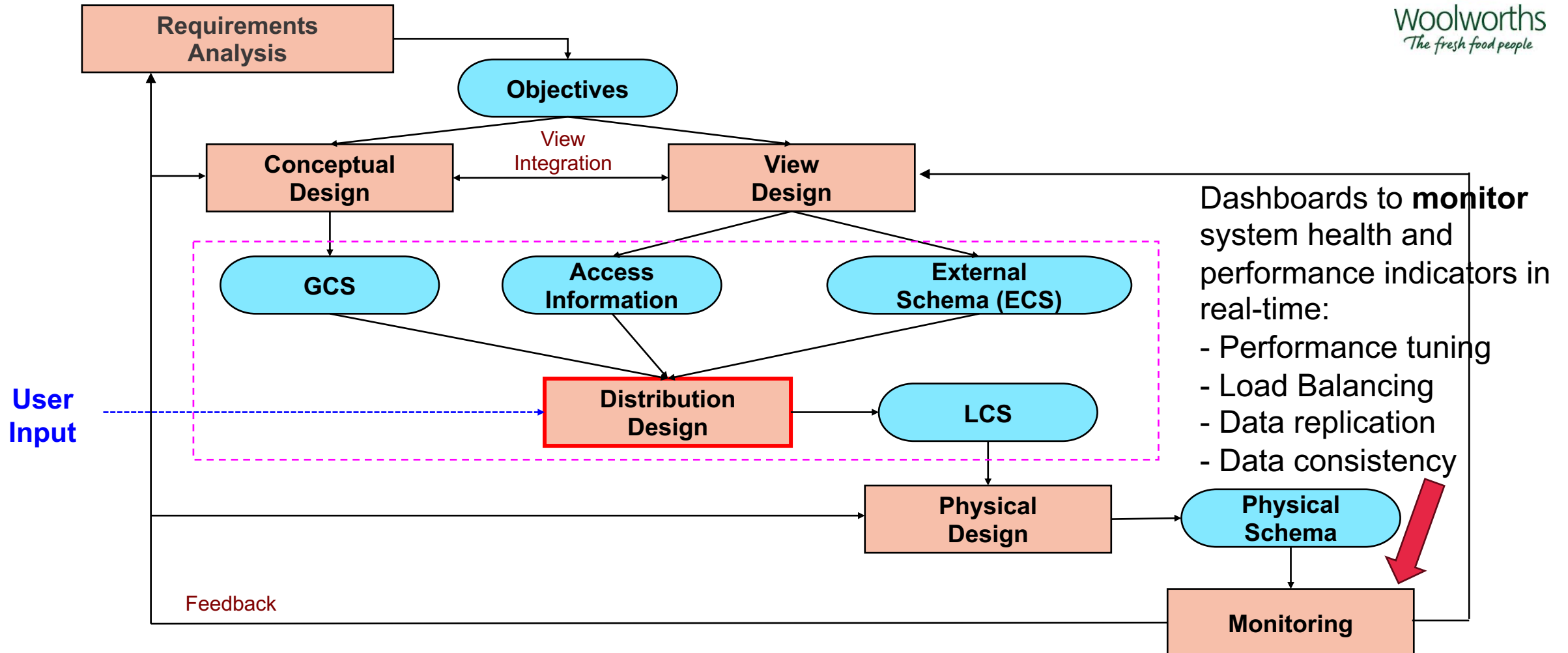
- Choosing the hardware and storage solutions for different sites.
- Indexing strategy for efficient data retrieval.
- Clustering data to optimise disk usage.

Woolworths would need a robust physical design to ensure quick access to inventory data to handle high transaction volumes, especially during peak shopping seasons.



Feedback

An Example of Top-Down Design Process (6)



Fragmentation & Allocation

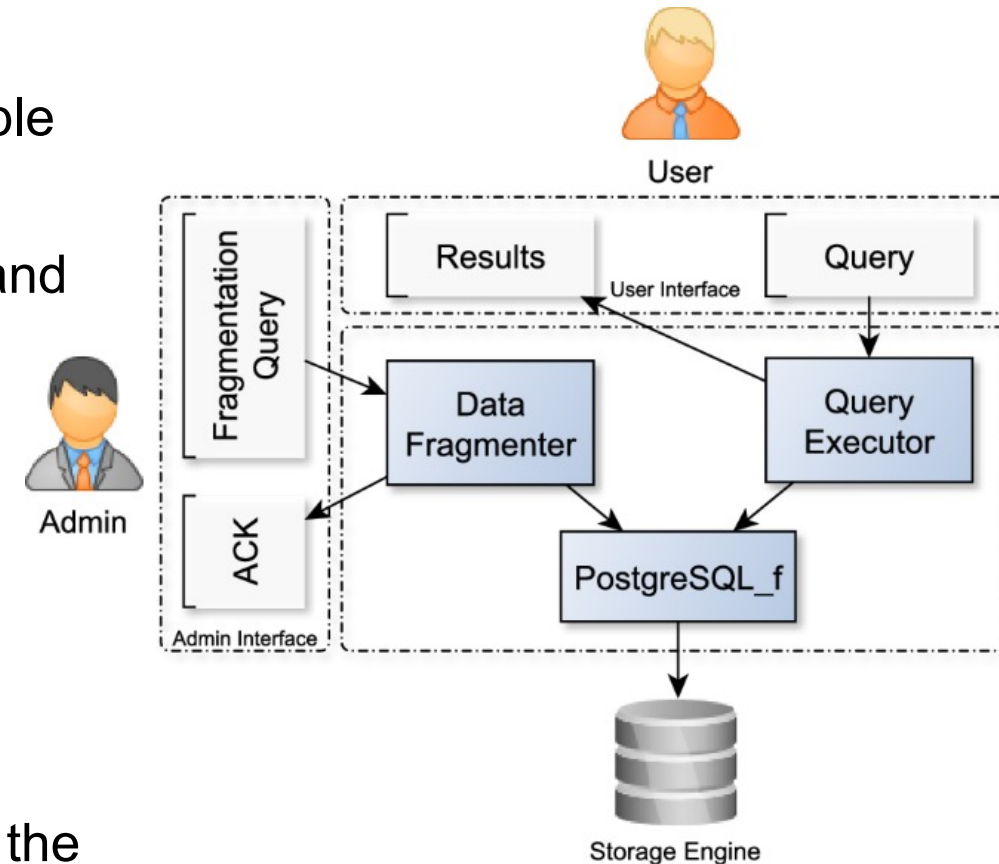
Fragmentation - How to split the data?

- The process of dividing the database into smaller multiple parts or sub-tables is called fragmentation.
- The smaller parts or sub-tables are called **fragments** and are stored at different locations.
- Data fragmentation should be done **ONLY** when reconstruction is possible.
 - The restoration: **UNION** or **JOIN** operations.
- Types: Horizontal, Vertical and Mixed fragmentations.

Allocation - Where should each fragment go?

- The process to decide **where** exactly you want to store the data in the database.

<https://link.springer.com/article/10.1007/s10619-018-7250-4>



Example about locations of data

Student(id, name, age, location,...)

Assume 75% of queries are about St Lucia and 25% of queries are about Gatton

Q1: select *
from Student
where location = "St Lucia" and ...

Q2: select *
from Student
where location = "Gatton" and ...

Can a student be in both "St Lucia" and "Gatton"?

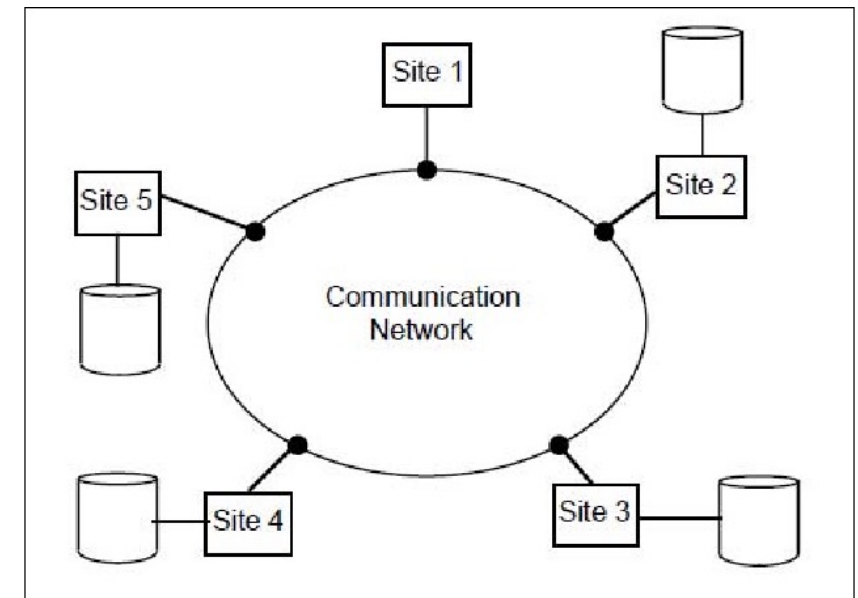
If there are two sites: St Lucia and Gatton
We can have fragmentation $F = \{F1, F2\}$

$F1 = \sigma_{\text{location}=\text{"St Lucia"}} \text{ Student}$
 $F2 = \sigma_{\text{location}=\text{"Gatton"}} \text{ Student}$

Can this solution completely fragment the table?

Why Fragmentation ?

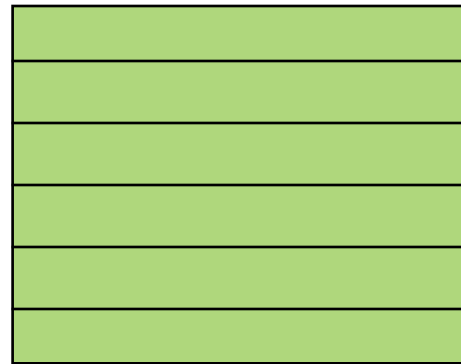
- Application views are subsets of relations - **a fragment is a subset of tuples in a relation**
- If there is no fragmentation
 - **Option 1:** Entire table is at one site - high volume of remote data accesses, or
 - **Option 2:** Entire table is replicated - problems in executing updates
- Better performance from parallel execution of query, since sub-queries can operate on fragments, in **parallel!**
- Sites may **own data**, so fragmentation is sometimes forced



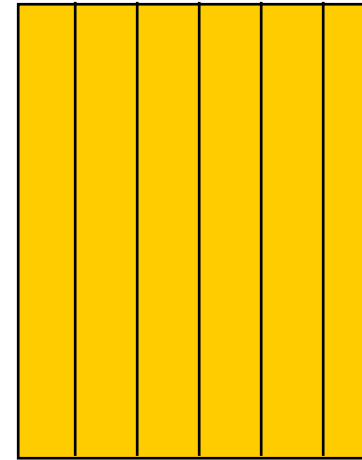
Drawbacks of Fragmentation

- **Application views** do not require **mutually exclusive fragments** - queries on multiple fragments will suffer **performance degradation** (costly joins and unions)
- Semantic **integrity control** - checking for dependency of attributes that belong to several fragments may require access to multiple sites
- Questions:
 - How much should we fragment?
 - How to resolve conflicting requirements from different applications?

Types of Fragmentation



Horizontal



Vertical

*...how to derive fragments
in SQL?
...it's also possible to have
mixed fragmentation*

Horizontal Fragmentation:

- **Primary fragmentation:** Segmenting a single table in a row-wise manner (using predicates on a single table only)
 - E.g. `SELECT * FROM Employee WHERE Salary >=50000; -> Emp_F1 & Emp_F2`
- **Derived fragmentation:** Fragmentation that is being derived from primary relation (using predicates with joined foreign relations)
 - E.g. `SELECT * FROM Emp_F1 WHERE DepID IN (SELECT DepID FROM Department)`

Horizontal Fragmentation: Example

PERSON

<u>NAME</u>	ADDRESS	PHONE	SAL
John Smith	44, Here St	3456 7890	34000
Alex Brown	1, High St	3678 1234	48000
Harry Potter	99, Magic St	9976 4321	98000
Jane Morgan	87, Riverview	8765 1237	65800
Peter Jennings	65, Flag Rd	9851 1238	23980

How can this table be partitioned?

Can you define a predicate for each of these fragments?

PERSON-FRAGMENT1

<u>NAME</u>	ADDRESS	PHONE	SAL
John Smith	44, Here St	3456 7890	34000
Peter Jennings	65, Flag Rd	9851 1238	23980

PERSON-FRAGMENT2

<u>NAME</u>	ADDRESS	PHONE	SAL
Alex Brown	1, High St	3678 1234	48000
Harry Potter	99, Magic St	9976 4321	98000
Jane Morgan	87, Riverview	8765 1237	65800

Horizontal Partitioning Techniques

Good Data Distribution for Different Queries

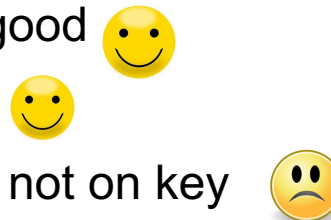
Round robin partitioning

- Distribute data evenly
- Good for scanning the whole table
- Not good for point or range queries



Hash partitioning

- Distribute data evenly if the hash function is good
- Good for point queries on key and joins
- Not good for range queries and point queries not on key



Range partitioning

- Good for some range queries on partition attribute
- Need to select good vector to avoid data/execution skew



Good Fragmentation?

Student(id, name, age, location,...)

Example 1:

F1 = $\sigma_{\text{age} > 35}$ Student
F2 = $\sigma_{\text{age} < 20}$ Student

Example 2:

F1 = $\sigma_{\text{age} > 35}$ Student
F2 = $\sigma_{\text{age} < 40}$ Student

- Is it complete?
- Does it have duplicates?

Fragmentation Properties

- $R \Rightarrow F = \{F_1, F_2, \dots, F_n\}$
- Horizontal fragmentations should not only ensure that the design matches **access patterns** but also meet the following three properties:

CDR property:

- **C**ompleteness: $\forall t \in R, \exists F_i \in F: t \in F_i$
- **D**isjointness: $\forall F_i, F_j \in F, i \neq j \Rightarrow F_i \cap F_j = \phi$
- **R**econstruction: $R = \bigcup_{i=1, \dots, n} F_i$

...why disjoint?

...if a fragmentation is complete, is it also reconstructable?

Desirable Fragmentation

Primary horizontal fragmentation is defined using simple conditions

To ensure **completeness** and disjointness

- **Approach 1:** manually check if the predicates meet the properties

$F1 = \sigma_{age > 35} \text{ Student}$

$F2 = \sigma_{age \leq 35} \text{ Student}$

$F1 = \sigma_{location = \text{"St Lucia"}} \text{ Student}$

$F2 = \sigma_{location = \text{"Gatton"}} \text{ Student}$

- **Approach 2:** automatically generate predicates with such properties (*minterm predicates approach*)

Minterm Predicates Approach

Simple predicate: $P_i = a_j \theta \text{ value}$; where $\theta = \{<, >, =, \dots\}$

Given a set of simple predicates $P = \{P_1, P_2, \dots, P_n\}$

Minterm predicate $M_i = P_1^* \wedge P_2^* \wedge \dots \wedge P_n^*$

- P_i^* is either P_i or $\neg P_i$
- Thus, there can be **2^n** such predicates in total

Eliminate useless ones (some are invalid)

- Outcome: $M = \{M_1, M_2, \dots, M_k\}$
- Each **minterm** predicate defines a fragmentation

More discussions in next week's tutorial

- Examples
- Why can **minterm** predicates meet the properties of **completeness** and **disjointness**?

What is a M_i for $i = 1 \dots n$?

A Quick Example: *Minterm* Predicates Approach

Imagine a UQ's Student table with the attributes Country and Status.
We want to fragment this table based on two simple predicates:

- **P1**: Country = 'International'
- **P2**: Status = 'Full-time'

The minterm predicates approach would create **minterms** by considering all combinations of these predicates and their negations:

- M1: P1 AND P2 (Students are from the International and full-time)
- M2: P1 AND NOT P2 (Students are from the International and NOT full-time)
- M3: NOT P1 AND P2 (Students are NOT International but are full-time)
- M4: NOT P1 AND NOT P2 (Students are NOT International and NOT full-time)

```
CREATE TABLE M1 AS
SELECT * FROM Student
WHERE Country = 'International' AND Status = 'Full-time';
```

```
CREATE TABLE M2 AS
SELECT * FROM Student
WHERE Country = 'International' AND Status <> 'Full-time';
```

```
CREATE TABLE M3 AS
SELECT * FROM Student
WHERE Country <> 'International' AND Status = 'Full-time';
```

```
CREATE TABLE M4 AS
SELECT * FROM Student
WHERE Country <> 'International' AND Status <> 'Full-time';
```

Derived Horizontal Fragmentation - revisited

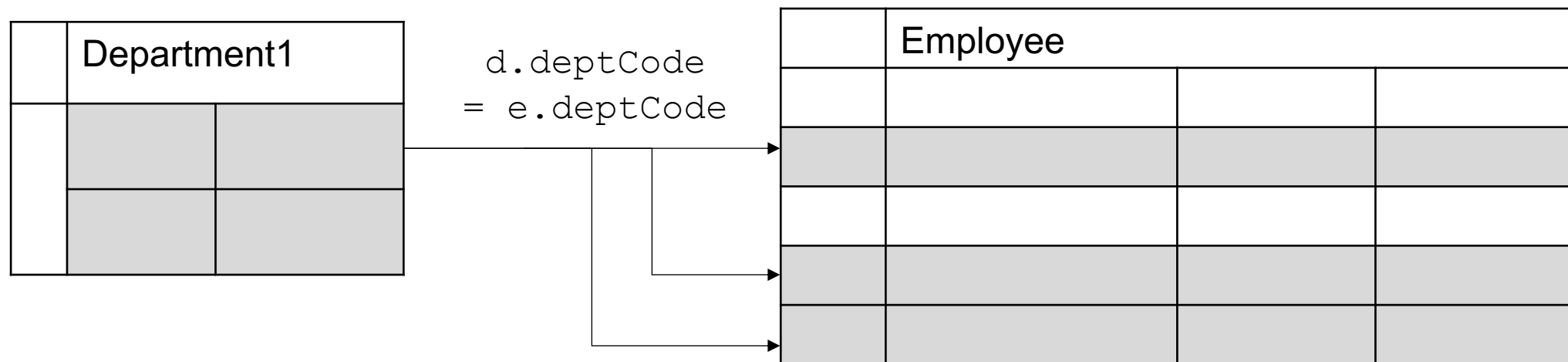
Employee (EmpId, Name, Age, DeptId,...)

Department (DeptId, State, Contact,...) → Departement1 (QLD), ...

Local Query:

```
SELECT e.* FROM Department1 d, Employee e
WHERE d.deptCode = e.deptCode;
```

Efficient Query?



Fragmentation Using *Semi-join* ⋈

- **Semi-join** operator is defined as: $R1 \bowtie R2 = \pi_{R1} R1 \bowtie R2$
- Consider two Relations **R** and **S** which can be fragmented by **Semi-join** as:
- $R \Rightarrow F = \{F_1, F_2, \dots\}, S \Rightarrow G = \{G_1, G_2, \dots\}, G_i = S \bowtie F_i$ (R is Department and S is Employee)
- **R** is called the **owner** of the fragmentation, and **S** is a **member**



SELECT * FROM Employee WHERE deptCode IN
(SELECT deptCode FROM Department1);

Fragmentation Using *Semi-join* ✕

Why do we need semi-join in data fragmentation?

Join operation is very expensive, we need to reduce the size of the data involved in a join operation as much as possible.

Semi-Join has Three kinds of applications:

1. In **Centralized Database** to reduce the quantity of reading of the data from the hard disk into the memory.
2. In **Distributed database Fragmentation** to define the fragments in the derived fragmentation process.
3. In **distributed query processing** to reduce the data communications from the different sites.

Questions:

- How to ensure completeness?
- How to ensure disjointness?
- How to reconstruct the original table?

Semi-join algorithm will be discussed in the coming weeks

Vertical Fragmentation: Example

PERSON

<u>NAME</u>	ADDRESS	PHONE	SAL
John Smith	44, Here St	3456 7890	34000
Alex Brown	1, High St	3678 1234	48000
Harry Potter	99, Magic St	9976 4321	98000
Jane Morgan	87, Riverview	8765 1237	65800
Peter Jennings	65, Flag Rd	9851 1238	23980

PERSON1

<u>NAME</u>	ADDRESS	PHONE
John Smith	44, Here St	3456 7890
Alex Brown	1, High St	3678 1234
Harry Potter	99, Magic St	9976 4321
Jane Morgan	87, Riverview	8765 1237
Peter Jennings	65, Flag Rd	9851 1238

PERSON2

<u>NAME</u>	SAL
John Smith	34000
Alex Brown	48000
Harry Potter	98000
Jane Morgan	65800
Peter Jennings	23980

...primary Key [NAME] is included in all fragments, why?

Vertical Fragmentation Properties

Given a table $R[A]$, $A = \{a_1, a_2, \dots, a_n\}$,

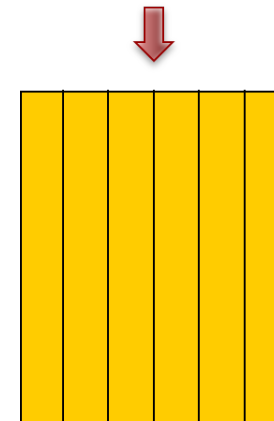
we define $A_i \subseteq A$, where

$$R[A] \Rightarrow R_i[A_i], \quad i = 1, \dots, m$$

Completeness: $\bigcup_{i=1, \dots, m} A_i = A$

Disjointness: $\forall i, j, \quad i \neq j \Rightarrow A_i \cap A_j = \phi ?$

- Disjointness does not guarantee reconstructability!
- One way is to duplicate key attributes in every fragmentation
- The reason to include Primary key in every vertical fragment is to maintain the **entity integrity** of the relation.



Vertical

How to Group Attributes based on the frequencies of distributed queries?

Affinity matrix

$R = \{\underline{k}, a1, a2, a3, a4, a5\}$

	a1	a2	a3	a4	a5
a1	-	-	-	-	-
a2	50	-	-	-	-
a3	45	48	-	-	-
a4	1	2	0	-	-
a5	0	0	4	75	-

The numbers in this matrix are the frequencies of queries.

Grouping Co-accessed Attributes

$R = \{\underline{k}, a1, a2, a3, a4, a5\}$

	a1	a2	a3	a4	a5
a1	-	-	-	-	-
a2	50	-	-	-	-
a3	45	48	-	-	-
a4	1	2	0	-	-
a5	0	0	4	75	-

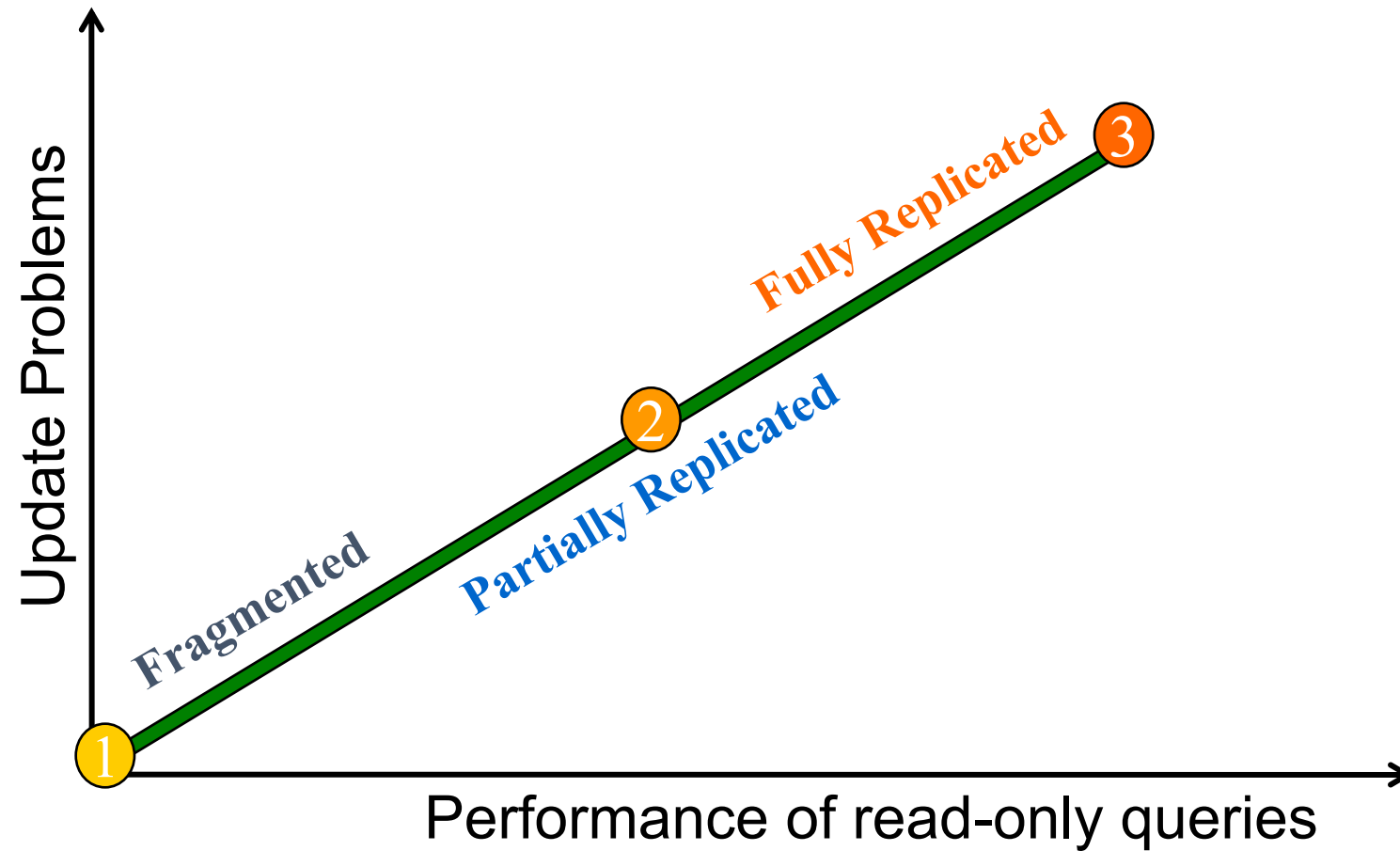
$R_1[k, a_1, a_2, a_3]$

$R_2[k, a_4, a_5]$

Fragments with Replication

- In a centralised system, it has a single copy of all data
- One way to distribute a database is to copy some or all data, possibly several times
 - Each copy is stored at a different node in the network
 - Query routed to “nearest” node
- Fragments can be replicated
 - Full replication or partial replication
- All copies of replicated data must be updated in a single transaction to maintain atomicity and consistency

Trade-off of Fragmentation with Replication



Allocation of Fragments to Sites

- Now data has been partitioned (and possibly replicated), where should each fragment go?
- Many issues to consider:
 - Where do **queries** originate?
 - What is the communication **cost**?
 - ✓ Size of answers? Relations?
 - What is the **storage capacity** and cost at **sites**?
 - ✓ Size of fragments?
 - What is the **processing power** at sites?
 - What is the query processing strategy?
 - ✓ How are **joins** done?
 - ✓ Where are answers collected?

What is fragment allocation?

Allocation is an Optimisation Problem

- Best placement of fragments and/or best number of copies to a set of sites in order to:
 - **Minimise** query response time
 - **Maximise** system throughput
 - **Minimise** some other costs, such as cost of storing and querying a fragment at a site, cost of updating a fragment at all sites, cost of data communication
- Subject to constraints
 - Available **storage**
 - Available **bandwidth**, **power**, etc.
 - Other, e.g., keep 90th percentile **response time** $< x$



What is an ideal X?

...what kind of queries?

Inputs to Fragment Allocation

- **Database information**
 - **Fragment Selectivity**: #tuples of a fragment F_i that need to be accessed in order to process a query q_j
- **Application information**
 - **Read access**: the number of read accesses that a query q_j makes to a fragment F_i during execution
 - **Update access**: the number of update accesses that a query q_j makes to a fragment F_i during execution
- **System information**
 - **Site information**
 - Knowledge of storage and processing capacity
 - **Network information**
 - Communication cost (channel capacities, distances, protocol overhead)

...how to find optimal allocation?

Allocation: Assigning Fragments to Sites

- **Input:**

- **Fragments:** $F=\{F_1, F_2, \dots, F_m\}$ **Sites:** $S=\{S_1, S_2, \dots, S_n\}$
- Typical **queries:** $Q=\{Q_1, Q_2, \dots, Q_k\}$ detailed with read/write information

- **Output :** an **allocation** can be represented by a group of mappings:

- $X_{ij} = 1$ if F_i is assigned to S_j ; otherwise $X_{ij}=0$

Total_cost = total_local_processing_cost + total_data_exchange_cost + total_stoarge_cost

- The allocation problem is to find an **optimal mapping** to minimize the total cost, with challenges: (1) **NP-complete**, (2) hard to estimate precise costs, (3) changing costs over time
- **Solutions: heuristics-based** with many simplifications (e.g., communication costs only), supporting dynamic adjustment

... What is the definition of cost?

SSOT (Single Source of Truth) property with DDB

- What is the **Single Source of Truth** (SSOT) problem?
- How do we manage it?
 - No duplicate data entries or version control issues
 - Timely data values at the right moment
 - Reduced time spent validating records and data types
 - Improved data warehouse and intelligence
 - Improved communication and productivity
- Example in **Healthcare** or SCM (**Supply-Chain Management**)
- Questions to consider for SSOT
 - “Where is the data from?”
 - “How old is the data point?”
 - “Is the data set complete and accurate?”
 - “What version are we looking at?”



Questions of DDB Design

- Fragmentation
 - Which tuples to group together in horizontal fragmentation?
 - How to check if a horizontal fragmentation is complete?
 - Which attributes to group together in vertical fragmentation?
 - How do we **insert data** when a **table** is **fragmented**?
 - What about database **constraints** when a **table** is **fragmented**?
 - What is the **Minterm** used for?
 - How **Semi-join** is used in fragmentation?
- Allocation
 - What is the definition of allocation?
 - What to optimize during the allocation?
 - How to formulate the optimization problem?
 - How to solve the optimization problem?

Summary of DDB Design

- There are different types of distributed design ([top-down](#) and [bottom-up](#)), but top-down design is mostly used in DDB design
- DDB has a [Global Conceptual Schema](#)
- [Centralized vs distributed](#) – understanding of main differences
- [Transparency](#) levels and problems associated with them
- Issues with provision of *full* scale DBMS functionality
- Data [fragmentation](#), [replication](#) and [allocation](#)
- Properties of Fragmentation: [Completeness](#), [Disjointness](#), [Reconstruction](#) (**CDR**)
- [Minterm](#) predicates are used to maintain CDR
- [Semi-Join](#) operator can be used in fragmentation
- [SSOT](#) is a property of DDB.